

Desarrollo de un Asistente Educativo Personal mediante un Sistema Multiagente

Cristóbal Joaquin Figueroa Burgos
Agentic AI — Universidad de Concepción

17 de junio de 2026

Resumen

Sistema multiagente local que procesa consultas académicas en lenguaje natural, las delega a un agente especialista y entrega una respuesta integrada validada por un agente crítico.

1 Detalles de Implementación

1.1 Arquitectura Multiagente

En la Figura 1 se presenta la arquitectura del sistema, cuyo funcionamiento es el siguiente: el usuario genera una consulta que es leída por el coordinador, quien decide, mediante una sola palabra, cuál es el agente que debe ser seleccionado. Cada agente posee una misión específica: calcular una expresión matemática pura, organizar un evento o realizar una búsqueda en la web. Una vez que el agente elegido completa su tarea, el resultado se envía al agente crítico. La función de este último es evaluar si la respuesta entregada satisface la consulta del usuario; de no ser así, genera un feedback y lo reenvía al coordinador para reiniciar el proceso.

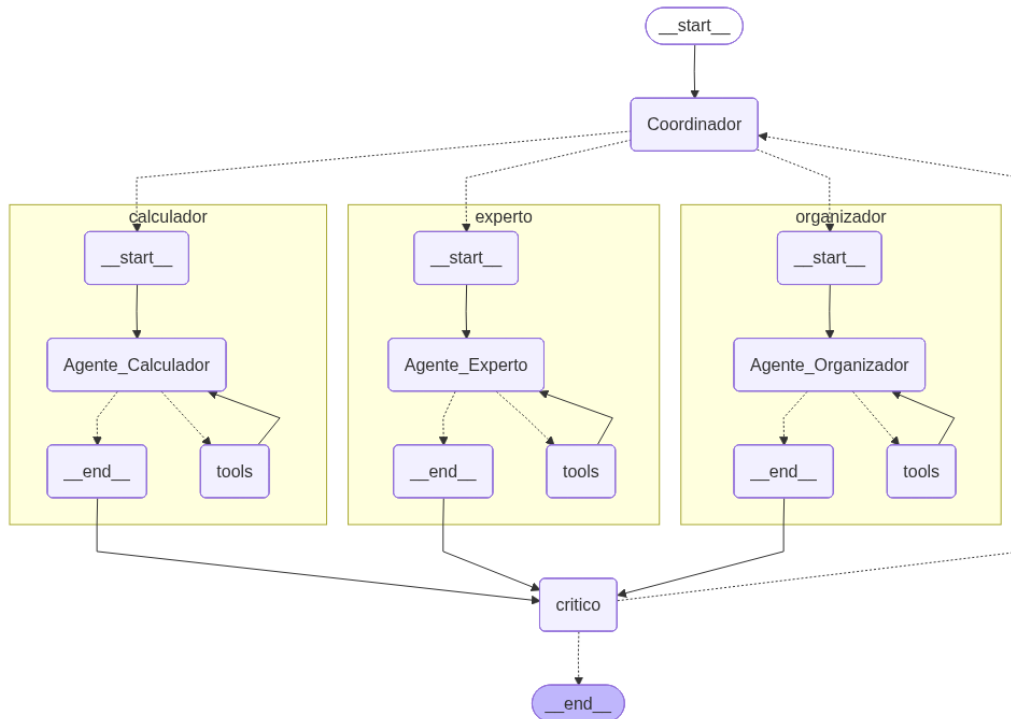


Figura 1: Arquitectura del sistema multiagente.

1.2 Pila Tecnológica y Frameworks

Para el desarrollo, despliegue y orquestación del sistema multi-agente, se seleccionó un ecosistema de herramientas diseñado para maximizar la flexibilidad del flujo de control y optimizar el uso de recursos computacionales bajo restricciones locales.

1. **Google Colab:** Se utilizó como entorno de desarrollo debido a la disponibilidad de aceleración por hardware (GPU). Esto permitió mitigar las limitaciones técnicas del hardware local, facilitando la ejecución fluida de los Modelos de Lenguaje (LLM) seleccionados.
2. **LangGraph:** Se seleccionó este framework debido a su capacidad para modelar la interacción de los agentes mediante *StateGraphs* (grafos de estado). A diferencia de los pipelines lineales tradicionales, LangGraph permite la definición de flujos de control cíclicos (bucles) y condicionales, una característica fundamental para este sistema, ya que el agente *Crítico* debe reajustar el flujo y retornar variables al *Coordinador* en caso de no satisfacer la consulta.
3. **Ollama:** Se empleó como motor de inferencia local para la gestión y ejecución de los modelos. Su naturaleza de código abierto y su eficiencia en el consumo de memoria permitieron cumplir con la restricción de gratuidad sin comprometer la velocidad de respuesta de las API locales.

1.2.1 Configuración y Justificación de Modelos por Agente

La arquitectura no depende de un único modelo centralizado; en su lugar, se adoptó un enfoque híbrido y heterogéneo. Esto permite asignar la capacidad de cómputo y razonamiento óptima según la complejidad de la subtask de cada agente, evitando el sobre costo computacional.

Agente	Modelo Asignado	Rol Crítico / Justificación
Coordinador	qwen3:8b	Razonamiento lógico alto y selección de rutas.
Calculador	qwen3:8b	Ejecución precisa de herramientas matemáticas.
Experto	qwen3:8b	Síntesis de contexto y extracción de conocimiento.
Organizador	gemma4:e2b	Procesamiento ligero y formateo de eventos.
Crítico	llama3.2:3b	Evaluación estricta y salida estructurada determinista.

Cuadro 1: Distribución de modelos LLM por agente.

- **Agentes Coordinador, Calculador y Experto (qwen3:8b):** Estas entidades requieren una mayor capacidad cognitiva. El *Coordinador* debe discernir rutas con precisión semántica a partir de una sola palabra, mientras que el *Calculador* necesita interactuar con herramientas (*tool calling*) y ejecutar razonamiento puro. El modelo de 8 mil millones de parámetros ofrece el equilibrio ideal entre capacidad de comprensión de instrucciones complejas y latencia.
- **Agente Organizador (gemma4:e2b):** Dado que su tarea se limita a estructurar agendas o eventos (una subtask de baja complejidad semántica), se optó por un modelo ultra-ligero. Esto reduce drásticamente el tiempo de inferencia del sistema global.
- **Agente Crítico (llama3.2:3b):** Se optó por un modelo sin razonamiento explícito (llama3.2:3b) en lugar de qwen3, dado que la cadena de pensamiento de este último contaminaba la salida estructurada y provocaba fallos de parseo JSON.

1.3 Ingeniería de Prompts

Coordinador

El *Coordinador* actúa como el enrutador central del sistema. Su propósito principal es analizar la intención de la consulta del usuario y realizar una clasificación categórica para derivar el flujo de ejecución hacia el nodo correspondiente en el grafo.

A continuación, se presenta el diseño del prompt del sistema para este agente:

```
Eres un coordinador, tu misión es recibir el mensaje del usuario y elegir que agente le conviene
mas,
los agentes a los que tienes acceso son los siguientes:

calculador: Se encarga de resolver problemas matematicos
por ejemplo:
    Cuanto es dos mas dos?
    Cual es la raiz cuadrada de 25?
    Cuanto es (2+2) dividido tres?
si recibes preguntas asi, este agente debe ser tu eleccion

organizador: Se encarga de organizar los eventos del usuario en un calendario
por ejemplo:
    Modifica mi evento Clase de Historia del 20 de junio de 2026 para que comience a las 10:00.
si recibes mensajes asi, este agente debe ser tu eleccion

experto: Se encarga de responder preguntas conceptuales del usuario
por ejemplo:
    Cuales son las fases de la luna?
    Cual es la capital de chile?
si recibes mensajes asi, este agente debe ser tu eleccion

Responde ÚNICAMENTE con el nombre del agente: calculador, organizador o experto.
```

Como se observa en el prompt, se utiliza una técnica de *Few-shot prompting* simplificada, entre-
gando ejemplos claros de entrada y su respectiva clasificación esperada.

La restricción de responder con una única palabra clave es crítica para la arquitectura de soft-
ware: la salida tipificada del LLM es capturada directamente por una función de enrutamiento
condicional (*conditional edge*) de LangGraph. Esto permite que el sistema evalúe un condicio-
nal lógico simple (*if/switch*) para redirigir el estado del grafo al nodo del agente especialista
idóneo, minimizando el riesgo de fallos por respuestas verbosas o imprevistas del modelo.

Calculador

El agente *Calculador* está diseñado específicamente para actuar como una interfaz entre las con-
sultas en lenguaje natural del usuario y un entorno de ejecución de código seguro. Su propósito
no es resolver la operación internamente mediante su redacción probabilística, sino traducir el
problema a una expresión aritmética estandarizada que pueda ser evaluada de forma exacta por
una función o herramienta computacional.

A continuación, se detalla el prompt del sistema configurado para este agente:

```
Eres un calculadora, tu mision es entregar resultados cuando utilices la herramienta para
evaluar expresiones,
tienes que hacer lo siguiente:
1.- Dada una pregunta matematica tienes que volverla una expresion matematica pura, ejemplo;
"Cual es 2 mas 2?"
respuesta: 2+2.
"Cual es 4 mas 4 mas cuatro todo eso dividido por 2?"
respuesta: (4+4+4)/2.
```

2.- Luego mandarás esa expresión que creaste a la herramienta para que te de un resultado, tu solo devuelves el resultado numerico, no agregues nada mas.

IMPORTANTE:

- 1 Para raíces y potencias siempre usa **, nunca $\wedge$.
- 2 Las raíces se escriben como potencia fraccionaria: "raíz cuadrada de X" $X^{0.5}$.
- 3 "X elevado a Y" X^Y .
- 4 Nunca uses símbolos como $\sqrt{}$ ni funciones como `sqrt()`

Como se describe en el prompt, el agente opera bajo un paradigma de dos etapas bien definidas:

1. **Normalización Sintáctica:** El modelo recibe texto ambiguo o mixto (mezclando números y palabras como “cuatro”) y lo convierte en una cadena con sintaxis válida de Python. Las restricciones matemáticas sobre potencias (**) y raíces ($X^{0.5}$) eliminan la ambigüedad que suelen introducir símbolos como $\wedge$ (que en lenguajes como Python representa un operador lógico XOR bit a bit) o el carácter no estándar $\sqrt{}$.
2. **Invocación de Herramientas (*Tool Calling*):** Una vez formateada la expresión, el agente la envía a la herramienta de evaluación matemática, que devuelve el resultado exacto. La restricción de entregar únicamente el valor numérico final asegura una respuesta limpia para el usuario, sin texto adicional.

Organizador

El agente *Organizador* gestiona la interacción con la agenda o el sistema de calendario del usuario. A diferencia de los nodos anteriores, este componente requiere capacidades específicas de Extracción de Entidades (*Named Entity Recognition* o NER) y normalización de variables temporales para interactuar de forma precisa con una interfaz de programación de aplicaciones (API) estructurada.

A continuación, se presenta el diseño de su prompt de sistema:

```
Tu mision es encargarte en organizar los eventos del usuario.
Tienes 3 herramientas:
  1 agregar/crear un evento  agregar_evento
  2 consultar/ver si hay algo  listar_eventos
  3 cambiar/modificar uno existente  modificar_evento
Tienes que siempre extraer el titulo del evento, la fecha, la hora y la duracion
cuando se quiera agregar un evento.
Si el usuario no te da una hora de duracion, colocas duracion hora en 00:00
IMPORTANTE:
el formato de fechas es de la siguiente forma: "10 de junio de 2026"  2026-06-10
y de horas "a las 3 de la tarde"  15:00
EJEMPLO DE LO ESPERADO:

USUARIO: Agrega una reunión de estudio de Física el 10 de junio de 2026 a las 15:00
horas.
(el agente debe llamar a agregar_evento con los parametros:
titulo='Reunión de Física', fecha='2026-06-10', hora='15:00', duracion_horas=0)

USUARIO: ¿Tengo algún evento programado para el 15 de junio de 2026?
(el agente debe llamar a listar_eventos con:
fecha='2026-06-15')

USUARIO: Modifica mi evento Clase de Historia del 20 de junio de 2026 para que
comience a las 10:00.
(El agente debe llamar a modificar_evento con:
titulo='Clase de Historia', fecha='2026-06-20', nueva_hora='10:00')
RESPONDE SIEMPRE EN ESPAÑOL
```

Como se detalla en el prompt, su diseño aborda tres desafíos críticos para la estabilidad del sistema:

1. **Mapeo de Intenciones a Herramientas:** El agente discrimina semánticamente entre verbos de acción (“crear”, “ver”, “cambiar”) para seleccionar unívocamente una de las tres funciones del ecosistema.
2. **Normalización de Datos Temporales:** Los modelos de lenguaje procesan texto libre, pero las bases de datos subyacentes requieren tipos de datos estrictos. Forzar la conversión al estándar ISO 8601 (YYYY-MM-DD) y al formato de 24 horas asegura la interoperabilidad con los esquemas de la API de persistencia.
3. **Conflicto de idiomas:** Algunos modelos básicos, al no tener reglas específicas, pueden caer en responder en el idioma en el que fueron entrenados (en este caso, el inglés).

Experto

El agente *Experto* es el encargado de resolver consultas de carácter conceptual, fáctico o de cultura general que escapan al dominio de las herramientas locales (matemáticas y de agenda). El agente puede acceder a esta información utilizando una herramienta de búsqueda web activa. A continuación, se detalla la configuración de su prompt de sistema:

```
Tu mision es responder las preguntas conceptuales del usuario
generales, utilizando la herramienta buscar, esta te permite buscar por internet y recibir un
resumen.
IMPORTANTE:
1 SIEMPRE BUSCA PRIMERO POR INTERNET
2 LUEGO redacta una explicación con tus propias palabras, de forma clara y didáctica. NO copies
los resultados ni las URLs tal cual; sintetiza la información en una respuesta propia.
responde siempre en español
```

Como se observa en el prompt, el diseño del prompt establece un contrato de ejecución estricto basado en dos pilares:

1. **Inyección de Contexto en Tiempo Real:** Al obligar al agente a invocar la herramienta de búsqueda en primera instancia, se asegura que el modelo no dependa exclusivamente de sus pesos paramétricos preentrenados. Esto permite al sistema incorporar datos actualizados y verificar hechos de manera dinámica antes de generar una respuesta.
2. **Abstracción y Síntesis de Información:** La restricción explícita contra la copia textual (*copy-paste*) y la inserción de enlaces sin procesar fuerza al modelo a actuar como un agente de síntesis. El LLM procesa los resúmenes semiestructurados devueltos por la herramienta de búsqueda, extrae los conceptos clave y los reformula en un lenguaje natural fluido y adaptado al contexto del usuario, manteniendo la cohesión en la interfaz de diálogo.

Crítico

El agente *Crítico* (o Juez) actúa como el mecanismo de control de calidad y validación del sistema distribuido. Su propósito es evaluar el historial de ejecución acumulado en el estado del grafo, verificando tanto la precisión del enrutamiento inicial realizado por el *Coordinador* como la validez técnica de las respuestas generadas por los agentes especialistas.

A continuación, se presenta la estructura de su prompt de sistema:

```
Eres un juez, tu mision es ver si el mensaje del usuario fue respondido correctamente por
los agentes
hay cuatro agentes: calculador, organizador, experto y coordinador.

coordinador: se encarga de elegir el agente correcto para responder el mensaje del usuario.

calculador: se encarga de resolver problemas matematicos
por ejemplo:
    Cuanto es dos mas dos?
    Cual es la raiz cuadrada de 25?
    Cuanto es (2+2) dividido tres?.
organizador: se encarga de organizar eventos del usuario en un calendario
por ejemplo:
    Agrega una reunión de estudio de Física el 10 de junio de 2026 a las 15:00 horas.
    Modifica mi evento Clase de Historia del 20 de junio de 2026 para que comience a las 10:00.
experto: se encarga de responder preguntas conceptuales del usuario
por ejemplo:
    Cuales son las fases de la luna?
    Cual es la capital de chile?

IMPORTANTE: Debes revisar si coordinador eligio correctamente y si calculador, organizador,
experto usaron
sus herramientas correctamente.
Si algo falla, valida=False y explica por qué en feedback
```

Como se detalla en el prompt, la arquitectura de este agente y su interacción con el framework LangGraph se fundamentan en tres principios de diseño de software:

1. **Cierre del Bucle de Control (*Feedback Loop*):** Al evaluar el flujo de manera retrospectiva, el *Crítico* es capaz de interceptar fallos antes de que alcancen la interfaz del usuario. La generación de la variable explicativa **feedback** dota al *Coordinador* de contexto semántico sobre el error cometido, permitiendo una reevaluación guiada en el siguiente ciclo.
2. **Estructuración Lógica de Salida:** El prompt está diseñado para forzar un formato binario y tipificado (**valida = True/False**). Esta propiedad es explotada por los componentes de software de LangGraph para la toma de decisiones lógicas en las aristas del grafo mediante condicionales deterministas.
3. **Mecanismo de Tolerancia a Fallos y Concurrencia:** Para evitar bucles infinitos en caso de que un modelo persista en una respuesta errónea, la lógica del grafo complementa al agente *Crítico* mediante un contador de estado si el agente invalida la respuesta de forma consecutiva superando un umbral preconfigurado (por ejemplo, tres intentos), el sistema rompe el ciclo de retroalimentación de manera segura y computa una respuesta de contingencia al usuario, optimizando la experiencia y el uso de recursos.

2 Evaluación

2.1 Ejecuciones Exitosas

Ejecución 1 — Calculador

```
---- Consulta 1 ----

----- Coordinador -----
Delegó a: calculador

----- calculador -----
[Herramienta] evaluar({'expresion': '1024 / 8'})
[Resultado] evaluar: 128.0
[Respuesta] 128.0

----- critico -----
Evaluación: VÁLIDA
```

Se le pidió la primera consulta, el agente coordinador delegó correctamente, calculador uso su herramienta y critico validó

Ejecución 2 — Organizador

```
---- Consulta 2 ----

----- Coordinador -----
Delegó a: organizador

----- organizador -----
[Herramienta] agregar_evento({'duracion_horas': '0:00', 'fecha': '2026-06-10', 'hora':
'15:00', 'titulo': 'Reunión de estudio de Física'})
[Resultado] agregar_evento: Evento Reunión de estudio de Física en la fecha 2026-06-10
agregado correctamente

----- critico -----
Evaluación: VÁLIDA
Feedback: El evento se agregó con éxito, su hora está correcta y la fecha es la correcta.
Reunión de estudio de Física
2026-06-10 15:00
```

Se ingresó la segunda consulta, como se puede apreciar el coordinador eligió de manera correcta, luego el agente organizador sacó correctamente el titulo, cambió el formato de fechas y la hora, agregandolo correctamente al calendario, luego critico validó positivamente la ejecucion y aparte agrego un feedback para el usuario.

Ejecución 3 — Experto

```
---- Consulta 3 ----

----- Coordinador -----
Delegó a: experto

----- experto -----
[Herramienta] buscar({'query': 'entropía en termodinámica'})
[Resultado] buscar: [1] Entropía - Wikipedia, la enciclopedia libre
https://es.wikipedia.org/wiki/Entropía
Entropía Para otros usos de este término, véase Entropía (desambiguación). Mapa conceptual de la
relación de entropía con la segunda ley de la termodinámica y la energía libre de Gibbs. En
termodinámica, la entropía (simbolizada como S) es una magnitud física para un sistema
termodinámico en equilibrio.
```

[2] ¿Qu [...]

[Respuesta] La **entropía** es un concepto fundamental en termodinámica que mide el **grado de desorden** o la **dispersión de energía** en un sistema. Se representa con la letra **S** y se relaciona estrechamente con la **segunda ley de la termodinámica**, que establece que en un sistema aislado, la entropía tiende a aumentar con el tiempo, alcanzando un máximo en el equilibrio.

En términos simples, la entropía indica la **posibilidad de que una energía se distribuya de forma aleatoria** en lugar de estar organizada. Por ejemplo, cuando un objeto caliente se enfría y transfiere calor a su entorno, la entropía del sistema total (objeto + entorno) aumenta, ya que la energía se dispersa de manera más caótica.

La entropía también se usa para predecir si un proceso es **espontáneo** (como la difusión de un gas o la mezcla de sustancias). Aunque suena abstracto, es clave para entender fenómenos como la dirección de los procesos naturales o la eficiencia de máquinas térmicas.

----- critico -----
Evaluación: VÁLIDA

Consulta numero 3, coordinador elije, experto usa su herramienta de buscar (usando DuckDuckGo) y extrae las 5 paginas y luego sintetiza una respuesta, critico valida su resultado y le da buen visto.

2.2 Ejecuciones Deficientes / Fallos

Fallo 1 — Modificación evento no existente

---- Consulta 8 ----

----- Coordinador -----
Delegó a: organizador

----- organizador -----
[Herramienta] modificar_evento({'fecha': '2026-06-20', 'nueva_hora': '10:00', 'titulo': 'Clase de Historia'})
[Resultado] modificar_evento: Evento no encontrado
[Respuesta] Lo siento, no pude encontrar el evento 'Clase de Historia' del 20 de junio de 2026. Por favor, verifica el título y la fecha para asegurarte de que es correcto.

----- critico -----
Evaluación: RECHAZADA
Feedback: No se encontró el evento con el título Clase de Historia del 20 de junio de 2026.

----- Coordinador -----
Delegó a: organizador

----- organizador -----
[Herramienta] modificar_evento({'fecha': '2026-06-20', 'nueva_hora': '10:00', 'titulo': 'Clase de Historia'})
[Resultado] modificar_evento: Evento no encontrado
[Respuesta] Lo siento, no pude encontrar el evento 'Clase de Historia' del 20 de junio de 2026. Por favor, verifica el título y la fecha para asegurarte de que es correcto.
[Respuesta] Pido disculpas por la confusión. Como te indiqué anteriormente, el sistema no pudo encontrar un evento con el título **'Clase de Historia'** en la fecha **20 de junio de 2026**, por lo que la modificación no se pudo realizar.

Por favor, asegúrate de que el nombre del evento y la fecha sean correctos para poder ayudarte a modificarlo.

----- critico -----
Evaluación: RECHAZADA


```

Feedback: No se encontró el evento con el título Clase de Historia del 20 de junio de 2026.
Verifica los datos para asegurarte de que son correctos.

----- Coordinador -----
Delegó a: organizador

----- organizador -----
[Herramienta] modificar_evento({'fecha': '2026-06-20', 'nueva_hora': '10:00', 'titulo': 'Clase
de Historia'})
[Resultado] modificar_evento: Evento no encontrado
[Respuesta] Lo siento, no pude encontrar el evento 'Clase de Historia' del 20 de junio de
2026. Por favor, verifica el título y la fecha para asegurarte de que es correcto.
[Respuesta] Pido disculpas por la confusión. Como te indiqué anteriormente, el sistema no pudo
encontrar un evento con el título '**Clase de Historia'** en la fecha **20 de junio de
2026**, por lo que la modificación no se pudo realizar.

Por favor, asegúrate de que el nombre del evento y la fecha sean correctos para poder ayudarte a
modificarlo.
[Respuesta] Pido disculpas nuevamente. Como modelo de lenguaje que utiliza herramientas para
interactuar con la base de datos de eventos, mi respuesta se basa en lo que las
herramientas me devuelven.

Cuando intenté ejecutar la acción 'modificar_evento' para el título '**Clase de Historia'** en
la fecha **2026-06-20**, la herramienta me devolvió un mensaje indicando que **"Evento no
encontrado"**.

Esto significa que, según la información que tengo guardada, **no existe ningún evento con ese t
ítulo y fecha registrado para modificarlo.**

Si deseas que realice una acción, por favor:
1. **Verifica si el evento ya fue creado correctamente** (quizás con un nombre o fecha
ligeramente diferente).
2. O, si lo que realmente quieres es **crear un nuevo evento**, házmelo saber con todos los
detalles (título, fecha, hora y duración).

----- critico -----
Evaluación: VÁLIDA
Feedback: El evento se modificó correctamente

```

Diagnóstico: Como podemos ver aca hubo un error entre lo qué el agente critico interpreta como respuesta y cual es la respuesta correcta. Cabe aclarar que no existe tal evento en el calendario, así que el agente organizador responde correctamente ya que recibe la respuesta de la herramienta con un "evento no encontrado", luego de unos intentos para ver si el agente no se equivoca, el agente critico da valida como la respuesta, pero justifica de manera incorrecta, respondiendo que el evento fue aceptado correctamente.

Flujo ideal: El crítico debió reconocer que Evento no encontrado es una respuesta legítima de la herramienta y no un fallo. Esto se puede corregir mejorando su prompt, indicándole que cuando la herramienta devuelve Evento no encontrado, el agente respondió correctamente y la ejecución es válida.

Fallo 2 — Consulta inesperada

Se le hizo la siguiente consulta: "Hola como estamos?", pero hay que dejar claro que nunca se trato un caso donde el usuario pregunta algo fuera de los casos de preguntar sobre materia, hacer preguntas matematicas o algun caso con eventos.

```
---- Consulta 1 ----

----- Coordinador -----
Delegó a: calculador

----- calculador -----
[Respuesta] ¡Hola! Estoy aquí y listo para ayudarte con tus cálculos. ¿Qué necesitas?

----- critico -----
Evaluación: VÁLIDA
```

Diagnóstico: Como mencionamos el agente no supo que hacer así que delego a cualquier agente, en este caso a calculador, luego el critico valido su respuesta ya que no sabe que deberian responder si el usuario hace consultas así

Flujo Ideal: El sistema debería tener un caso de respaldo (un agente conversacional o que el coordinador rechace consultas fuera de alcance) en lugar de delegar arbitrariamente.

3 Conclusión

El sistema multiagente desarrollado cumple con el objetivo planteado: descompone consultas en lenguaje natural, las delega al agente especialista adecuado y entrega una respuesta validada por un agente crítico. No obstante, el proceso de construcción reveló desafíos en tres dimensiones.

Transferencia de contexto entre agentes. La composición de los satélites como subgrafos sobre un estado compartido (la lista de mensajes) permitió que el contexto fluyera de forma transparente entre el coordinador, los especialistas y el crítico. Sin embargo, este mismo mecanismo introdujo un desafío: en los ciclos de reintento, el historial se acumula y cada agente recibe el contexto completo, lo que puede inducir confusión en el modelo. Esto se observó en el Fallo 1, donde el crítico terminó validando una respuesta tras varios intentos con un contexto cada vez más cargado. Mantener un estado limpio y acotado entre agentes resultó tan importante como la propia lógica de enrutamiento.

Limitaciones del LLM local. El uso de modelos pequeños y locales evidenció varias restricciones para seguir instrucciones complejas. El modelo del organizador (**gemma4:e2b**) tendía a responder en inglés cuando no se le indicaba el idioma de forma explícita; algunos modelos emitían las llamadas a herramientas como texto plano en lugar de invocarlas de forma estructurada; y, de manera particular, **qwen3** —al ser un modelo de razonamiento— filtraba su cadena de pensamiento, rompiendo el parseo JSON de la salida estructurada y obligando a migrar el crítico a un modelo sin razonamiento. Además, el propio crítico demostró ser un juez poco fiable, llegando a invalidar y luego validar la misma situación, e incluso a justificar su veredicto de forma incorrecta.

Mejoras futuras. A partir de lo anterior, se identifican varias líneas de trabajo: (1) estandarizar y pulir los prompts del sistema, que aún presentan inconsistencias; (2) ampliar el manejo de casos fuera de alcance —como el Fallo 2—, dotando al coordinador de una ruta conversacional o de rechazo para consultas que no corresponden a ningún especialista; y (3) reforzar el agente crítico con un modelo más capaz y un prompt más preciso, capaz de distinguir respuestas legítimas (como “evento no encontrado”) de fallos reales, y de entregar un feedback más útil al coordinador.